1 Estimation engine: the Kalman filter and RTS smoother

This section documents the engine's recursive estimator. It is a variable-dimension extended Kalman filter (EKF) with an optional prior-anchored iterated (IEKF) relinearisation pass, robust gating of post-fit residuals, and a backward Rauch–Tung–Striebel (RTS) smoother for post-processing. The state vector $\boldsymbol{x}$ is the uncombined PPP / PPP-RTK parameterisation of the state-vector-layout section of the Foundations document: receiver position (and optionally velocity), receiver clock and per-system time offsets, zenith wet troposphere and gradients, per-satellite slant ionosphere, receiver and satellite code biases, and one float ambiguity per tracked carrier. The dimension changes from epoch to epoch as satellites rise and set and as ambiguities are reset. `filter/StateManager` owns the layout and the dynamic key→index maps. The filter operates on that state and assumes no fixed indices (`src/core/filter/KalmanFilter.cpp`; its sparse products are in `KalmanKernels.cpp`, the post-fit screen in `KalmanPostFitQc.cpp`, the statistical residual QC in `KalmanResidualQc.cpp` and the outlier responses in `KalmanOutlierResponse.cpp`). The network estimator does not use this filter (the network-mode document); it solves per-epoch normal equations and links epochs by carried priors.

The notation throughout is $\boldsymbol{F}$ for the state-transition matrix, $\boldsymbol{Q}$ for the discrete process-noise covariance, $\boldsymbol{R}$ for the measurement-noise covariance, $\boldsymbol{S}$ for the innovation covariance, $\boldsymbol{K}$ for the Kalman gain, and $\boldsymbol{\nu}$ for the innovation (measurement minus predicted measurement). Predicted (a-priori) and updated (a-posteriori) quantities use the tilde/hat and $k|k-1$ / $k|k$ conditioning convention fixed in the notation-and-conventions section of the Foundations document.

1.1 Time update

State transition. The transition matrix $\boldsymbol{F}(\Delta t)$ for epoch interval $\Delta t = t_k - t_{k-1}$ is identity plus a small number of coupling blocks, assembled by `StateManager::getTransitionMatrix` (`src/core/filter/StateManagerProcessNoise.cpp`). The kinematic-integrator off-diagonal couplings are

$$\boldsymbol{F}[\,\boldsymbol{r},\boldsymbol{v}\,] = \Delta t\,\boldsymbol{I}_3, \qquad \boldsymbol{F}[\,c\mathrm{d}t,\ c\dot{\mathrm{d}}t\,] = \Delta t, \qquad \boldsymbol{F}[\,c\delta t^{(g)},\ c\dot{\delta t}^{(g)}\,] = \Delta t, \tag{1}$$

present only when the corresponding rate state exists (`hasVelocity()`, clock-rate, per-system rate). A fourth coupling appears when the constant-velocity slant-iono model is armed (`estimation.enable_iono_rate`, default off): a per-satellite level/rate integrator

$$\boldsymbol{F}[\,I^i,\ \dot{I}^i\,] = \Delta t \qquad \text{for every non-frozen-pivot satellite,} \tag{2}$$

which gives $I^i(k{+}1) = I^i(k) + \Delta t\,\dot{I}^i(k)$. Every other state has $\boldsymbol{F} = \boldsymbol{I}$ and is a random walk with constant mean: ambiguities, zenith wet delay, gradients, code and satellite biases, any inter-system offset without a rate, and (without `enable_iono_rate`) the per-satellite ionosphere.

Under the BSSD (single-difference) parameterisation some states are unobservable, because their $\boldsymbol{H}$ columns are zero: the receiver clock, the phase clock (`getPhaseClockIndex`), the inter-system offsets, and the frozen-pivot per-satellite ionosphere (`getFrozenPivotSatellites`). Their rows in Equation (1) are suppressed. After propagation their cross-covariances are set to zero and their diagonal variances are kept (the `freeze_state` block in `src/core/filter/KalmanFilter.cpp`).

The iono level/rate coupling of Equation (2) is *not* disabled under BSSD. The slant-iono level stays observable under single-differencing, since only the pivot column is zero, so every satellite other than the frozen pivot keeps its integrator. The frozen pivot is skipped. Its rate is held at $(0, 0)$

and its process noise is suppressed, so the coupling cannot move the pivot level away from the SD datum.

The propagation itself is the standard EKF time update

$$\boxed{\begin{aligned}\tilde{\boldsymbol{x}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\hat{\boldsymbol{x}}_{k-1|k-1}, &(3)\\ \tilde{\boldsymbol{P}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\boldsymbol{P}_{k-1|k-1}\,\boldsymbol{F}(\Delta t)^{\mathrm{T}} + \boldsymbol{Q}(\Delta t), &(4)\end{aligned}}$$

implemented in `KalmanFilter::predict`. A non-positive Δt (a duplicate or out-of-order epoch) does nothing: the routine returns before Equation (4).

Process-noise model. The discrete process noise $\boldsymbol{Q}(\Delta t)$ is built from continuous spectral densities by `StateManager::getProcessNoiseMatrix`, a short driver over one function per block of $\boldsymbol{Q}$ (`StateManagerProcessNoise.cpp`); the family blocks read their rule and spectral-density key from the family table (the declared state families), which the run-start log prints. Two block structures appear. Pure random-walk states have a scalar discrete variance increment

$$\boldsymbol{Q}_{ii} = q_i\,\Delta t, \tag{5}$$

where q_i is the continuous variance rate (m^2/s) read from `filter.process_noise.*`. Representative entries are zenith wet troposphere (`troposphere_q`), the receiver code bias (`code_bias_q`), the satellite *code* bias (`sat_bias_q`, applied to `StateManager::sat_bias_indices_`), the satellite *phase* bias (`sat_phase_bias_q`, a separate density on the separate `sat_phase_bias_indices_` map), phase clock (`phase_clock_q`), and the float ambiguity (`ambiguity_q`). The three satellite blocks (code bias, phase bias and clock, one function over the family table) set $\boldsymbol{Q}$ to zero for a frozen-pivot satellite, as do the ambiguity and ionosphere blocks. The frozen state is held at (value, variance) $= (0, 0)$ as the datum, and any process noise would move it off that datum. The ambiguity density defaults to a numerical floor (10^{-12}), not a physical value. Per-key overrides are supported for reseeding individual arcs. The per-satellite slant ionosphere uses a random walk whose zenith density is mapped to the slant by the squared obliquity factor,

$$\boldsymbol{Q}_{ii} = \frac{q_{\mathrm{iono}}}{\sin^2\theta^i}\,\Delta t, \qquad \sin^2\theta^i \text{ floored at } 10^{-2}\ (\theta \approx 5.7^\circ), \tag{6}$$

with q_{iono} = `ionosphere_q` and optional per-satellite overrides driven by the adaptive-iono scheme (off by default). The rule applies on every run that carries slant-ionosphere states (`GnssConfig::carriesSlantIonosphere`: uncombined processing outside the baseline modes), PPP included. Under the constant-velocity model of Equation (2) this random walk is used only for satellites without a rate state. Otherwise the level/rate pair takes the CWNA block of Equation (7) below, with the acceleration PSD scaled by obliquity, $q = q_{\dot{\mathrm{iono}}}/\sin^2\theta^i$, where $q_{\dot{\mathrm{iono}}}$ = `process_noise.iono_rate_psd` (an acceleration PSD, $\mathrm{m}^2/\mathrm{s}^3$, default 10^{-10}). The adaptive-monitor override enters this branch as the *dimensionless* ratio $q_{\mathrm{ov}}/q_{\mathrm{iono}}$, which multiplies the rate PSD. The Sage–Husa storm response therefore acts on $\dot{I}^i$, not on the level.

Coupled position/velocity, clock/clock-rate, inter-system offset/rate, and, under `enable_iono_rate`, slant-iono level/rate pairs use the continuous white-noise-acceleration (CWNA, integrated-random-walk) discretisation. For a coupled pair $(\xi, \dot{\xi})$ driven by acceleration PSD q,

$$\boldsymbol{Q}_{[\xi,\dot{\xi}]} = \begin{bmatrix} \frac{1}{3}\,q\,\Delta t^3 & \frac{1}{2}\,q\,\Delta t^2 \\ \frac{1}{2}\,q\,\Delta t^2 & q\,\Delta t \end{bmatrix}, \tag{7}$$

with q given by `acceleration_psd_q` (position/velocity, the 3×3 block-diagonal extension of Equation (7) over the ECEF axes), `clock_rate_q`, `system_time_rate_offset_q`, and the obliquity-scaled `iono_rate_psd`$/\sin^2\theta^i$ respectively. When the velocity state is present the rate-free position density `position_q` is unused. Baseline (double-difference) mode reduces $\boldsymbol{Q}$ to the position block, a per-ambiguity random-walk term, and an optional differential-ZWD entry $\boldsymbol{Q}_{ii} = \sigma^2 \Delta t$ governed by `baseline.differential_tropo_process_noise_m_per_sqrt_s`.

Scope: the recursive estimator only. These densities apply to the *recursive* estimator. The network estimator does not run a `KalmanFilter`. `getTransitionMatrix` and `getProcessNoiseMatrix` are called only from `KalmanFilter::predict` / `predictAided` and from the smoother's `Application::capturePriorSnapshot`. The network solves per-epoch normal equations for corrections to each station's linearisation point. It links epochs with explicit information-form carry priors of effective variance $\mathrm{var}_{\mathrm{eff}} = \mathrm{var} + q\,\Delta t$, not through $\boldsymbol{Q}(\Delta t)$ (the network-mode document). There the satellite-side densities `sat_clock_q`, `sat_phase_bias_q` and `sat_code_bias_q` are read as those carry rates. They can also be set through the network-scoped `network.process_noise` class table, which is always active. Its per-class fallbacks equal the legacy `filter.process_noise.*` defaults, so an unmodified config gives the same values either way.

Aided time update. When inertial aiding is active, the aiding source (`aiding/ImuAidingSource`) mechanises the nominal position, velocity and attitude (strapdown) over the integrated interval and returns a predict contribution: a 15×15 INS error-state transition $\boldsymbol{\Phi}$ and noise $\boldsymbol{Q}_{\mathrm{ins}}$, with error order $[\delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$, with the full-state indices it owns. `KalmanFilter::predictAided` writes each contribution over an identity $\boldsymbol{F}$ and over the baseline $\boldsymbol{Q}$ built for the integrated interval (`spliceAndPropagate`, `KalmanKernels.cpp`). The write is an assignment on the rows the contribution owns, so their baseline process noise is not counted twice, and the means of states no contribution owns are not propagated. The covariance recursion of Equation (4) then runs on the full state, followed by the symmetrisation $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$. With no valid contribution `predictAided` calls the GNSS-only `KalmanFilter::predict` and gives a byte-identical result.

1.2 Measurement update

The measurement update (`KalmanFilter::update`) is a *batch* update: all observation rows of the epoch are processed together, not one at a time. A sparse-$\boldsymbol{H}$ optimisation speeds up the four products that involve the design matrix ($\boldsymbol{HP}$, $\boldsymbol{HPH}^{\mathrm{T}}$, $\boldsymbol{KH}$ and $\boldsymbol{H}\,\delta\boldsymbol{x}$). It agrees with the dense computation: entries of $\boldsymbol{H}$ at or below 10^{-12} are dropped from the row lists (`src/core/filter/KalmanKernels.cpp`).

Given the design matrix $\boldsymbol{H}$ from the linearisation of the code observation equation and phase observation equation of the Foundations document (position rows $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$, the linearised geometric range there), the innovation $\boldsymbol{\nu}$ (`measurement.innovations`), and the measurement-noise covariance $\boldsymbol{R}$ (a full matrix when supplied, otherwise the diagonal $\boldsymbol{R} = \mathrm{diag}(\sigma^2_{r,i,s})$ of the stochastic noise model there), the innovation covariance and gain are

$$\boldsymbol{S} = \boldsymbol{H}\,\tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}, \tag{8}$$

$$\boldsymbol{K} = \tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}}\,\boldsymbol{S}^{-1}. \tag{9}$$

The gain is not formed by inverting $\boldsymbol{S}$. $\boldsymbol{S}$ is symmetric, so the engine factorises it once with an $\mathrm{LDL}^{\mathrm{T}}$ decomposition and obtains $\boldsymbol{K} = \left(\boldsymbol{S}^{-1}\boldsymbol{H}\tilde{\boldsymbol{P}}\right)^{\mathrm{T}}$ from one solve against $\boldsymbol{H}\tilde{\boldsymbol{P}}$ (`ldlt.solve(HP)`.

`transpose()`). If the factorisation fails it falls back to a complete-orthogonal (pseudo-inverse) solve. The state update is

$$\delta \boldsymbol{x} = \boldsymbol{K}\,\boldsymbol{\nu}, \qquad \hat{\boldsymbol{x}}_{k|k} = \tilde{\boldsymbol{x}}_{k|k-1} + \delta \boldsymbol{x}. \tag{10}$$

Joseph-form covariance update. The covariance update uses the *Joseph* form, not the short form $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\tilde{\boldsymbol{P}}$:

$$\boxed{\boldsymbol{P}_{k|k} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\tilde{\boldsymbol{P}}_{k|k-1}\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}.} \tag{11}$$

The Joseph form keeps the covariance symmetric and positive semi-definite for any gain. That property is needed because the robust paths of Section 1.4 use a gain that is not the optimal one. The result is symmetrised again, $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$, to remove round-off. The factor $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})$ is built from the sparse $\boldsymbol{K}\boldsymbol{H}$ product by adding one on the diagonal. The same kernel, with the explicit gain $\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^{\mathrm{T}}\boldsymbol{S}^{-1}$, is the static routine `josephPseudoUpdate` (`KalmanKernels.cpp`). It is reused for the pseudo-observation constraints applied through `applyPseudoObservation` (the GIM ionosphere prior and the a-priori ZWD constraint) and for the IEKF pass of Section 1.3.

Post-fit residuals. After the update the post-fit residuals and their (diagonal) covariance are

$$\boldsymbol{r} = \boldsymbol{\nu} - \boldsymbol{H}\,\delta \boldsymbol{x}, \qquad S_{r,ii} = R_{ii} - \left(\boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}\right)_{ii}, \tag{12}$$

By default only the diagonal of S_r is formed. The full symmetrised $S_r = \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}$ is formed and stored when two conditions hold: the off-diagonal-R kernel is enabled (`enable_full_r_kernel`, default off), *and* a dense $\boldsymbol{R}$ was supplied (BSSD and baseline-DD epochs, whose $\boldsymbol{R}$ carries SD/DD cross-covariance). The redundancy and w-test QC of Section 1.4 then account for correlated observations through S_r^{-1}. The diagonal, and so Equation (13), is the same either way. The standardised residual used for gating (Section 1.4) is

$$s_i = \frac{r_i}{\sqrt{\max\{\, S_{r,ii},\, R_{ii},\, 10^{-12} \,\}}}. \tag{13}$$

The denominator is floored at the *working* R_{ii}, the inflated noise used in $\boldsymbol{R}$, not the formal noise. This is a deliberate compromise. When an ambiguity absorbs the measurement (double-differenced phase with an ambiguity gain near one), the rigorous denominator $S_{r,ii}$ shrinks toward $R\,\boldsymbol{S}^{-1}R$, and residuals near zero would register as outliers of many σ. The strictly correct standardisation uses $S_{r,ii}$ alone. The floor gives up that rigour for stability, and the source code says so.

1.3 Iterated relinearisation (IEKF)

When the initial position is far from the truth, one linearisation of the geometric-range relation of the Foundations document per epoch may not be enough. When `processing.relinearize_geometry` is enabled, and the run is not tightly-coupled IMU, the engine performs a prior-anchored iterated update (a Gauss–Newton, or iterated EKF, step) after the primary update.

Anchor. The iteration is anchored to the *stored pre-update prior* $(\boldsymbol{x}^{-}, \boldsymbol{P}^{-})$ that the primary update Equations (4) and (11) used. It is captured only when relinearisation is armed (`getLastUpdatePriorX` / `getLastUpdatePriorP`). It is the prior *after* any in-epoch ambiguity augmentation and outlier reset, so its dimension matches the posterior. It is not a snapshot from the top of the epoch loop.

Gauss–Newton step. Let $\boldsymbol{x}_i$ be the current linearisation iterate (the committed state at which `measurement` was linearised, with innovations $\boldsymbol{z} - h(\boldsymbol{x}_i)$ and $\boldsymbol{H} = \partial h/\partial \boldsymbol{x}\big|_{\boldsymbol{x}_i}$). The prior-anchored iterate (`KalmanFilter::updateFromPrior`, via `josephPseudoUpdate`) is

$$\boxed{\begin{aligned} \boldsymbol{\nu} &= \big(\boldsymbol{z} - h(\boldsymbol{x}_i)\big) - \boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i), && (14)\\ \boldsymbol{K} &= \boldsymbol{P}^- \boldsymbol{H}^{\mathrm{T}} \big(\boldsymbol{H}\boldsymbol{P}^- \boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}\big)^{-1}, \qquad \boldsymbol{x}^+ = \boldsymbol{x}^- + \boldsymbol{K}\,\boldsymbol{\nu}, && (15)\\ \boldsymbol{P}^+ &= (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\boldsymbol{P}^-\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}. && (16) \end{aligned}}$$

The offset term $-\boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i)$ in Equation (14) expresses the innovation about the fixed prior, which makes this a MAP (Gauss–Newton) iteration. $\boldsymbol{K}$ and $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^-$ are always computed from the same fixed prior $(\boldsymbol{x}^-, \boldsymbol{P}^-)$, so the covariance does *not* shrink a second time and no information is counted twice.

Firing condition. The driver `Application::runGeometryRelinearization` (`src/core/app/PppEpochUpdate.cpp`) fires the iteration when all of the following hold:

1. `processing.relinearize_geometry` is on and a prior was stashed at the primary update;
2. the position change since the primary update exceeds a data-dependent threshold, `relinearize_dpos_threshold_phase_m` (default 1 m) when phase is present, or `relinearize_dpos_threshold_code_only_m` (default 100 m) for code-only epochs;
3. the per-run cap `relinearize_max_count` (-1 = unlimited) is not exhausted.

The inner loop recomputes the geometry and re-applies Equations (14) to (16) against the *fixed* exclusion set from the primary update. It runs at most five iterations and stops when the position step falls below 10^{-4} m. GNSS geometry is nearly linear at the metre level, so one pass usually converges, and the threshold of item 2 means the iteration rarely fires. The tightly-coupled IMU configuration takes a single legacy *naive* second pass, re-applying the measurement against the posterior, because error-state attitude and bias feedback conflicts with re-deriving the update from the prior.

1.4 Innovation and residual gating

The engine performs *no* χ^2 test on the innovation covariance. Outlier rejection (`KalmanFilter::validatePostFitResiduals`, a driver over named stages in `KalmanPostFitQc.cpp`) tests the standardised *post-fit* residuals s_i of Equation (13) against a fixed scalar threshold. Robust median-based detectors for pivot slips supplement it. The stages run in this order: the residual and statistical reports, the coherent pivot-offset detectors, the non-pivot multi-band slip detector, the Huber inflation pass, the gate (the σ threshold or the Baarda w-test), the response to the gated row and the complete-reset backstop. A detector that rejects a group ends the pass before the later stages. The two top-level thresholds are hardcoded `GnssConfig` defaults (`residual_threshold_` $= 6.0$ and `bad_meas_percent_` $= 0.5$ in `src/core/config/GnssConfig.cpp`). They are not exposed to JSON, so there are no config keys of those names. The per-row rejection threshold is $s_{\max} = 6.0$ (read at runtime via `getResidualThreshold`). The fraction that triggers a complete reset is 0.5 (`getBadMeasPercent`). The Baarda data-snooping gate below, which is exposed to JSON, can replace both the dispatched row and the criterion.

Per-row rejection. If $\max_i |s_i| > 6.0$, the worst row is handled according to its type (`KalmanOutlierResponse.cpp`). A phase outlier triggers an ambiguity reset, with BSSD pivot handling. A real slip moves the observation by an integer number of wavelengths, so a raw residual below $\lambda/4$ (`estimation.pivot_slip_min_cycles`, default 0.25, with `estimation.enable_subcycle_slip_suppression`, default on) is sub-cycle bias: the row is excluded from the pass and its ambiguity state is kept. A code outlier is excluded from the epoch. Correlated phase rows of the same satellite on other bands are reset with it. Among phase outliers of one constellation within 5 % of the maximum $|s|$, the satellite with the largest Melbourne–Wübbena deviation score is chosen.

Baarda data-snooping gate (optional override). When `estimation.enable_data_snooping` is set, the fixed-σ gate triple (gate stat, gate row, gate crit) $= (\max_i |s_i|, \arg\max, 6.0)$ is *replaced* in `validatePostFitResiduals` by the Baarda w-test. The test runs over *checkable* rows, those whose redundancy $r_i = S_{r,ii}/R_{ii}$ is at least 0.1. Rows with lower redundancy are skipped, because a blunder there goes into the estimate and not the residual. The row with the largest $|w_i|$ is selected, and the gate fires when $|w_i| >$ `estimation.data_snooping_critical` (default 3.29). Both the criterion *and* the dispatched row can therefore differ from the σ gate. The handling of the selected row by type is unchanged. The w-test statistic and redundancy are produced by the a-posteriori QC diagnostics layer (`computeResidualQc_` in `KalmanResidualQc.cpp`, run when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set): the variance factor $\hat{\sigma}_0^2 = \left(\sum_i v_i^2/R_{ii}\right)/\mathrm{dof}$, the per-row redundancy r_i, the w-test, and the minimal detectable bias (MDB). This layer accounts for a correlated R through S_r^{-1} when the full-R kernel is active (see Equation (12)).

Complete-reset backstop. If the number of rejected rows exceeds $0.5\times$ the original observation count (before any rows were removed), the filter covariance is re-inflated (`resetFilterCovariance`). This is a hard reset. In the smoother it marks a segment boundary (Section 1.5).

Robust pivot-slip detectors (BSSD). Before the per-row gate, single-difference rows are grouped by (pivot, signal) and screened for a disturbance common to the group, using median and median-absolute-deviation (MAD) statistics. Four detectors are the rows of one table (`kalman::kPivotDetectors`): phase per signal, phase pooled across bands, code per signal and code pooled across bands. A forced pivot handover fires when four conditions hold: the group is large enough (four rows in a per-signal group; three rows on two bands in a pooled one), at least half of its rows exceed the per-row threshold, the group median exceeds a floor (5 mm on phase, 0.30 m on code, above routine code multipath), and the MAD-to-median ratio is at most 0.4. A small ratio means a tight cluster, which indicates a common shift and not scattered noise. On phase per signal the median must also exceed the wavelength fraction $\lambda/4$ of `estimation.pivot_slip_min_cycles`, unless the cluster is tight (ratio at most 0.2), where only a common pivot disturbance explains several rows. A firing group requests a handover that excludes the satellites whose residuals disagree with the group median, and its rows are left out of the pass. Slips on non-pivot satellites across several bands are confirmed by integer-cycle consistency: $|r|/\lambda_{\mathrm{band}}$ must round to the same nonzero integer on each band.

Down-weighting (not gating). Two paths inflate $\boldsymbol{R}$ and do not reject. A persistent EWMA-bias term adds the squared mean bias to R_{ii} once an arc has warmed up (`estimation.persistent_bias_min_cycles`). A Huber post-fit scheme inflates R for sub-cycle rows whose $|s_i|$ exceeds the Huber knee (`estimation.phase_huber_sres_threshold`, default 6), iteratively on the post-fit residual (IRLS, the default) or once on the innovation, toggled by `enable_huber_r_inflation` (default on)

and `enable_huber_innovation` (default off). Both change the working $\boldsymbol{R}$ used in Equations (8) and (13). The standardised residual is therefore scaled by the working noise, not the formal noise.

1.5 Backward RTS smoother

In post-processing modes, enabling `processing.run_smoother` runs a fixed-interval Rauch–Tung–Striebel recursion [1] over the stored forward trajectory (`include/filter/Smoother.hpp`, `src/core/filter/Smoother.cpp`). The flag is forced off under `AppMode::RealTime`, because smoothing uses future data.

Forward-pass capture. The backward pass does not reconstruct the a-priori from $\boldsymbol{F}$ and $\boldsymbol{Q}$. The forward pass stores, per epoch, the one-step prediction $\tilde{\boldsymbol{x}}_{k|k-1}, \tilde{\boldsymbol{P}}_{k|k-1}$, the posterior $\hat{\boldsymbol{x}}_{k|k}, \boldsymbol{P}_{k|k}$, the matrix $\boldsymbol{F}_k$, and all core and dynamic key→index maps (`capturePriorSnapshot` / `capturePosteriorSnapshot` in `src/core/app/PppEpochRecords.cpp`). The order of capture makes the stored prior equal to $\boldsymbol{F}_k \hat{\boldsymbol{x}}_{k-1|k-1}$. The a-priori is recorded immediately after `predict()`, before any cycle-slip reseeding or measurement update. The a-posteriori is recorded *before* integer ambiguity resolution, so the smoother uses *float* ambiguities only. One edit follows the capture: a covariance-preserving integer repair of an ambiguity (the TurboEdit repair of the cycle-slip document) moves the ambiguity by a known $\mathrm{d}N\,\lambda$ in the frame of the stored prior. It is folded into $\tilde{\boldsymbol{x}}_{k|k-1}$ as a deterministic control input (`Application::foldSlipRepairsIntoPrior_`), so the backward pass does not read the jump as an innovation; a satellite that was also reset in the epoch is segmented instead (below).

Recursion. Initialised at the terminal posterior $\hat{\boldsymbol{x}}_{N|N} = \hat{\boldsymbol{x}}_{N|N}$, $\boldsymbol{P}_{N|N} = \boldsymbol{P}_{N|N}$, the backward recursion is

$$\boxed{\begin{aligned} \boldsymbol{G}_k &= \boldsymbol{P}_{k|k}\,\boldsymbol{F}_{k+1}^{\mathrm{T}}\,\tilde{\boldsymbol{P}}_{k+1|k}^{-1}, && (17)\\ \hat{\boldsymbol{x}}_{k|N} &= \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{G}_k\big(\hat{\boldsymbol{x}}_{k+1|N} - \tilde{\boldsymbol{x}}_{k+1|k}\big), && (18)\\ \boldsymbol{P}_{k|N} &= \boldsymbol{P}_{k|k} + \boldsymbol{G}_k\big(\boldsymbol{P}_{k+1|N} - \tilde{\boldsymbol{P}}_{k+1|k}\big)\boldsymbol{G}_k^{\mathrm{T}}. && (19) \end{aligned}}$$

The smoother gain $\boldsymbol{G}_k$ is computed by an $\mathrm{LDL}^{\mathrm{T}}$ solve, not an explicit inverse. The predicted covariance is symmetrised, then $\boldsymbol{G}_k = \big(\tilde{\boldsymbol{P}}_{k+1|k}^{-1}(\boldsymbol{P}_{k|k}\boldsymbol{F}^{\mathrm{T}})^{\mathrm{T}}\big)^{\mathrm{T}}$ via `ldlt.solve(...).transpose()`. The smoothed covariance Equation (19) is symmetrised after the update. If the factorisation fails, the step is skipped and epoch k keeps its forward posterior.

Variable-dimension reconciliation. Because $\dim \boldsymbol{x}_k \neq \dim \boldsymbol{x}_{k+1}$ in general, Equations (17) to (19) are restricted at each step to the *common-key subspace*, the states present at both k and $k{+}1$, found by intersecting the dynamic key→index maps (`collectCommonIndices`, less any keys in the handover exclusion set of the segment-boundary paragraph below). Let $\boldsymbol{\Pi}_k$ and $\boldsymbol{\Pi}_{k+1}$ select the common rows in each epoch's state space; the implemented state update is then

$$\hat{\boldsymbol{x}}_{k|N} = \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{\Pi}_k^{\mathrm{T}}\,\boldsymbol{G}_{k,C}\big(\boldsymbol{\Pi}_{k+1}\hat{\boldsymbol{x}}_{k+1|N} - \boldsymbol{\Pi}_k\tilde{\boldsymbol{x}}_{k+1|k}\big), \tag{20}$$

with the analogous projection on Equation (19). States outside the subspace keep their forward a-posteriori value. The subspace holds the position and the ZWD; the receiver clock, velocity, phase clock and gradients when both epochs have them; and the ambiguity, ionosphere, ionosphere-rate, receiver code-bias, satellite code-bias and inter-system offset keys present at both epochs. The clock-rate, inter-system rate and inertial states are not in it and keep their forward value. The

indexing needs care. $\tilde{\boldsymbol{x}}_{k+1|k}$, $\tilde{\boldsymbol{P}}_{k+1|k}$ and $\boldsymbol{F}_{k+1}$ are sized for epoch k's layout, because the predict step does not grow the state. They are addressed with epoch k's common indices. $\hat{\boldsymbol{x}}_{k+1|N}$ and $\boldsymbol{P}_{k+1|N}$ use epoch $k{+}1$'s layout.

Segment boundaries at resets and frame changes. Four mechanisms stop or restrict the backward flow of information at ambiguity resets, teardowns, and pivot handovers:

- A *complete* filter reset at $k{+}1$ (the covariance re-inflation of Section 1.4, recorded as `is_discontinuity`) is a hard segment boundary. It breaks the dynamic link $\tilde{\boldsymbol{x}}_{k+1|k} = \boldsymbol{F}\hat{\boldsymbol{x}}_{k|k}$, so $\boldsymbol{G}_k$ has no meaning. Backward flow stops and epoch k keeps its forward posterior.
- A *single-satellite* ambiguity reset is handled softly. That satellite's ambiguities are excluded from the common subspace for the $k \to k{+}1$ step. The re-seeded prior at $k{+}1$ has a large variance and carries no information backward, so smoothing it would only add noise. The exclusion matches by satellite prefix and is slightly broader than necessary. The extra states keep their forward values.
- A *dimension change* $\dim \tilde{\boldsymbol{x}}_{k+1|k} \neq \dim \hat{\boldsymbol{x}}_{k|k}$ is a hard boundary. Predict never grows the state, so a size mismatch means a long-gap teardown before predict (`Application::handleGap` prunes ambiguity, iono and sat-bias states before `predict()`). Across it, epoch k's indices would address the wrong states in the $k{+}1$ a-priori. Backward flow stops and epoch k keeps its forward posterior.
- A *BSSD pivot handover* between k and $k{+}1$ re-references one or more constellations in place. Each constellation is either reconciled or segmented, as described next.

BSSD pivot-handover reconciliation. When the pivot map changes between epochs ($\texttt{pivot_satellites}_k \neq \texttt{pivot_satellites}_{k+1}$) the smoothed posterior at $k{+}1$ is in the *new* pivot frame. The cached a-priori $\tilde{\boldsymbol{x}}_{k+1|k}$ was captured before the handover, in epoch k's layout, and is still in the *old* frame. The two cannot be compared directly in Equation (18). `Smoother::prepareHandoverFrame` makes them comparable, per constellation, on *copies* of the $k{+}1$ smoothed estimate. The reported $k{+}1$ result stays in its own frame. A handover that `PivotManager` recorded as *clean* (old and new pivots matching `clean_handovers`) is inverted back to the old pivot p_0, one signal group at a time, by the public static `Smoother::inverseReReferenceGroup`. This is the exact inverse of the forward re-reference. For a group with old-pivot datum row d,

$$x'(i) = x(i) - x(d), \quad x'(d) = 0, \qquad P'(i,\cdot) = P(i,\cdot) - P(d,\cdot), \quad P'(\cdot,j) = P(\cdot,j) - P(\cdot,d), \tag{21}$$

applied over *full* rows and columns, with the datum row and column set to zero last. Full rows are needed because the forward re-reference coupled the group to every other state. A signal group can be inverted only if the old pivot still has a live state on it *and* the forward step re-referenced that signal to the new pivot. Everything else is *segmented*: orphaned groups (the new pivot lacked the signal), disturbed or forced handovers, and the ionosphere and satellite-bias states of every handover constellation. Their keys are added to the `exclude_keys` set, which `collectCommonIndices` removes from the common subspace, and those states keep their forward posterior.

Static-position degeneracy. In static PPP the receiver position is nearly constant. The smoother gain for the position block then applies almost the same correction to every epoch, and the smoothed position is constant across epochs *by construction*. This is expected behaviour. The smoother improves the other states (troposphere, ionosphere, float ambiguities), and the position in kinematic runs, where it changes from epoch to epoch.

References

[1] H. E. Rauch, F. Tung, and C. T. Striebel. “Maximum likelihood estimates of linear dynamic systems”. In: *AIAA Journal* 3.8 (1965), pp. 1445–1450. DOI: 10.2514/3.3166.